

Báo cáo MLP Student Performance

Xây dựng mạng nơ-ron nhân tạo đa lớp MLP phân lớp Student Performance

Môn học: Học máy và Khai phá dữ liệu

Sinh viên:

Mã sinh viên:

Ngày thực hiện: 02/06/2026

1. Giới thiệu bài toán

Bài toán đặt ra là dự đoán kết quả cuối kỳ của sinh viên dựa trên các thông tin cá nhân, gia đình, thói quen học tập và hoạt động học tập. Đây là bài toán phân lớp đa lớp vì nhãn đầu ra GRADE có 8 lớp: 0 Fail, 1 DD, 2 DC, 3 CC, 4 CB, 5 BB, 6 BA, 7 AA.

2. Mô tả dữ liệu

Dataset Student performance.csv gồm 145 dòng và 33 cột. Cột STUDENT ID là mã sinh viên, không dùng làm đặc trưng huấn luyện. Các cột 1 đến 30 là đặc trưng đầu vào, trong đó câu 1-10 mô tả thông tin cá nhân, câu 11-16 mô tả thông tin gia đình, các câu còn lại mô tả thói quen và hoạt động học tập. Cột COURSE ID được dùng như một đặc trưng đầu vào. Cột GRADE là nhãn cần dự đoán.

Phân bố nhãn trong toàn bộ dữ liệu: 0 Fail: 8, 1 DD: 35, 2 DC: 24, 3 CC: 21, 4 CB: 10, 5 BB: 17, 6 BA: 13, 7 AA: 17.

Các đặc trưng dùng huấn luyện: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, COURSE ID.

3. Tiền xử lý dữ liệu

Chương trình bỏ cột STUDENT ID, tách X là các đặc trưng và y là nhãn GRADE. Dữ liệu được chia train/test theo tỷ lệ 80/20 bằng hàm tự cài đặt dựa trên numpy, có giữ phân bố lớp theo từng nhãn. Sau khi chia, chương trình fit Min-Max Scaling trên tập train và dùng cùng tham số đó để chuẩn hóa tập test. Cách làm này tránh rò rỉ dữ liệu từ tập test vào quá trình huấn luyện. Nhãn GRADE được mã hóa one-hot bằng hàm tự cài đặt.

Chuẩn hóa dữ liệu là cần thiết vì các đặc trưng có thang giá trị khác nhau. Khi đưa các đặc trưng về cùng khoảng giá trị, quá trình Gradient Descent ổn định hơn, giảm nguy cơ một số đặc trưng có giá trị lớn chi phối gradient.

4. Cơ sở lý thuyết MLP

MLP là mạng nơ-ron truyền thẳng gồm lớp đầu vào, một hoặc nhiều lớp ẩn và lớp đầu ra. Ở mỗi lớp, dữ liệu được nhân với ma trận trọng số, cộng bias, sau đó đi qua hàm kích hoạt. Trong chương trình này, lớp ẩn dùng ReLU: $\text{ReLU}(z) = \max(0, z)$. Lớp đầu ra dùng Softmax để chuyển logits thành xác suất cho 8 lớp.

Hàm mất mát dùng Cross Entropy cho phân lớp đa lớp. Với nhãn one-hot y và xác suất dự đoán p , loss được tính bằng trung bình của $-\sum(y * \log(p))$ trên toàn bộ mẫu. Gradient được tính bằng backpropagation. Trọng số và bias được cập nhật bằng Mini-batch Gradient Descent.

5. Thiết kế chương trình

File `mlp_student_performance.py` gồm các phần chính:

- ``load_data``: đọc dữ liệu CSV bằng pandas.
- ``split_features_labels``: bỏ ``STUDENT ID``, tách đặc trưng và nhãn.
- ``stratified_train_test_split``: chia train/test bằng numpy, không dùng sklearn.
- ``fit_minmax_scaler``, ``transform_minmax``: chuẩn hóa Min-Max tự cài đặt.
- ``one_hot_encode``: mã hóa nhãn one-hot tự cài đặt.
- ``MLPClassifierScratch``: cài đặt MLP từ đầu, gồm khởi tạo trọng số, forward propagation, ReLU, Softmax, Cross Entropy, backpropagation và cập nhật Gradient Descent.
- ``evaluate_model``: tính loss, accuracy và dự đoán.
- ``run_experiments``: chạy nhiều cấu hình mạng.
- ``plot_loss_curves``: vẽ biểu đồ loss.

Điều kiện dừng gồm: đạt số epoch tối đa, train loss nhỏ hơn ngưỡng cho trước, hoặc loss không cải thiện sau `patience = 20` epoch.

6. Thực nghiệm

Chương trình chạy 5 cấu hình MLP khác nhau:

Biểu đồ loss của cấu hình tốt nhất được lưu tại `loss_best_config.png`. Biểu đồ loss train của toàn bộ cấu hình được lưu tại `loss_all_configs.png`.

Cấu hình tốt nhất là Config 2 với cấu trúc [32], learning rate 0.01, test accuracy 0.3793 và test loss 1.8921. Cấu hình này được chọn vì có test accuracy cao nhất; khi hòa accuracy thì ưu tiên test loss thấp hơn.

Ma trận nhầm lẫn trên tập test (hàng là nhãn thật, cột là nhãn dự đoán):

	0	1	2	3	4	5	6	7
0:	1	1	0	0	0	0	0	0
1:	0	5	1	1	0	0	0	0
2:	0	4	1	0	0	0	0	0
3:	0	1	1	2	0	0	0	0
4:	0	1	0	0	0	1	0	0
5:	0	1	1	0	0	0	0	1
6:	0	1	0	2	0	0	0	0
7:	0	1	0	0	0	0	0	2

7. Nhận xét và đánh giá

Kết quả cho thấy thay đổi số lớp ẩn, số neuron, learning rate và số epoch ảnh hưởng trực tiếp đến khả năng học của mô hình. Cấu hình quá nhỏ thường có khả năng biểu diễn hạn chế, dễ underfitting nếu cả train accuracy và test accuracy đều thấp. Cấu hình lớn hơn có thể học tốt hơn trên tập train, nhưng với dataset chỉ có 145 mẫu, nếu chênh lệch train accuracy và test accuracy quá cao thì có dấu hiệu overfitting.

Learning rate lớn giúp mô hình học nhanh hơn nhưng có thể làm loss dao động. Learning rate nhỏ ổn định hơn nhưng cần nhiều epoch hơn. Điều kiện dừng sớm giúp tránh huấn luyện không cần thiết khi loss không còn cải thiện rõ rệt.

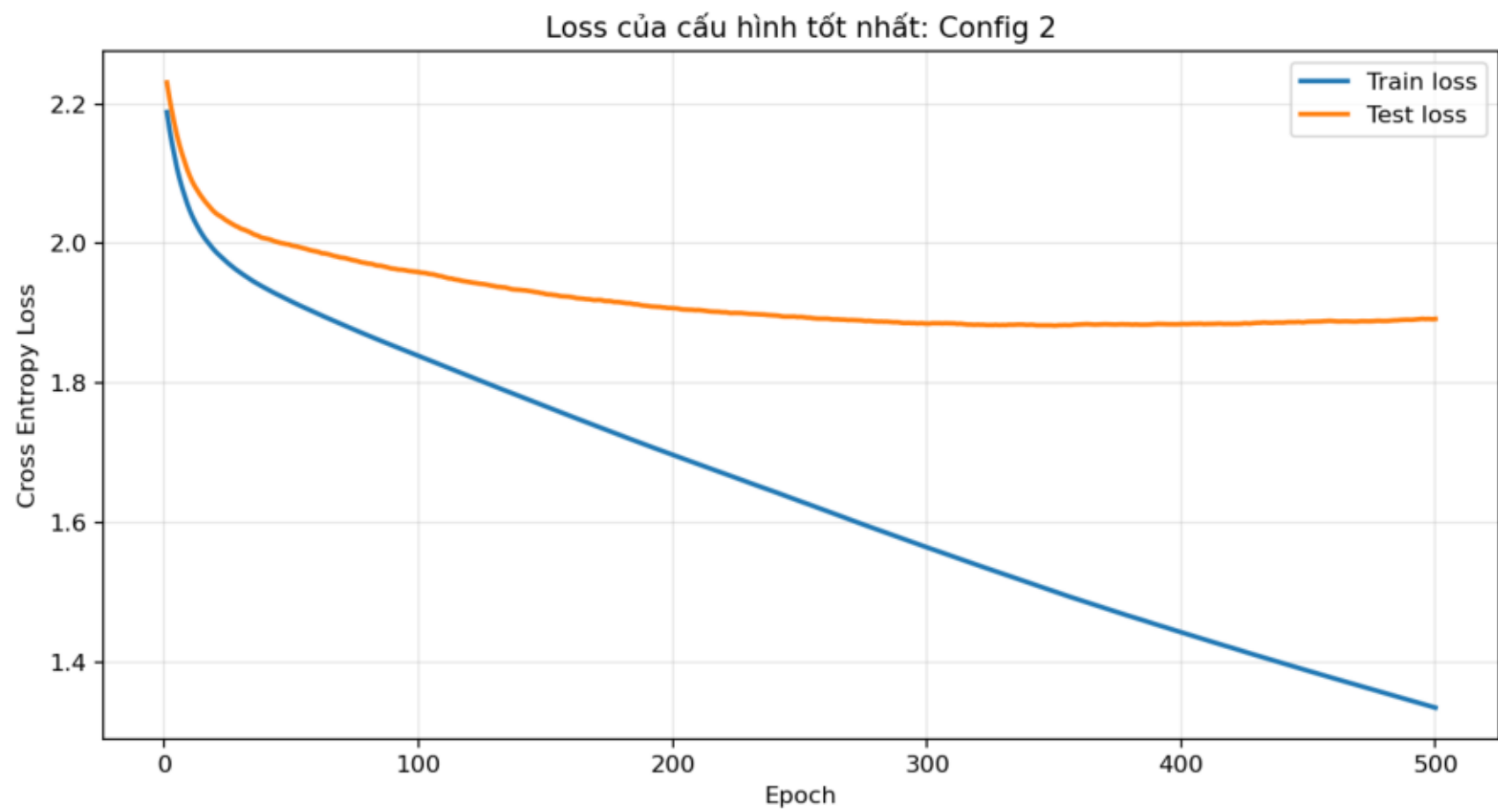
8. Kết luận

Bài làm đã cài đặt đầy đủ MLP từ đầu bằng numpy, không dùng sklearn, tensorflow, keras, pytorch hoặc classifier có sẵn. Chương trình đọc dữ liệu, tiền xử lý, chia train/test, huấn luyện 5 cấu hình, đánh giá bằng loss và accuracy, xuất bảng kết quả, vẽ biểu đồ loss và tạo báo cáo.

Hạn chế chính là dataset nhỏ, nhiều đặc trưng dạng categorical được mã số, nên mô hình có thể chưa tổng quát tốt. Các cải tiến có thể thử gồm k-fold cross validation, điều chỉnh learning rate, thêm regularization, thử one-hot cho các đặc trưng categorical và mở rộng tìm kiếm kiến trúc mạng.

Bảng so sánh kết quả - cấu hình tốt nhất: Config 2

STT	Cấu trúc	LR	Epoch	Epoch thực	Train loss	Test loss	Train acc	Test acc	Time(s)
1	[16]	0.01	500	500	1.5462	1.7636	0.4741	0.3103	0.172
2	[32]	0.01	500	500	1.3344	1.8921	0.5172	0.3793	0.251
3	[32, 16]	0.01	1000	1000	0.6109	2.0696	0.8621	0.3103	0.474
4	[64, 32]	0.005	1000	1000	0.8551	1.9898	0.7931	0.2759	0.548
5	[64, 32, 16]	0.001	1500	1500	1.6771	2.0351	0.4310	0.1724	0.902



Loss theo epoch cho 5 cấu hình MLP

